

Illustrative example for repSpat

```
library(sf)
library(ggplot2)
library(spdep)
library(Matrix)
library(MASS)
library(dplyr)
library(sp)
library(openxlsx)
library(adespatial)
library(igraph)
library(mclust)
library(SpatialExperiment)
library(Banksy)
library(scater)
```

Generate spatial domain

- Generating 'n' coordinates inside dim x dim grid

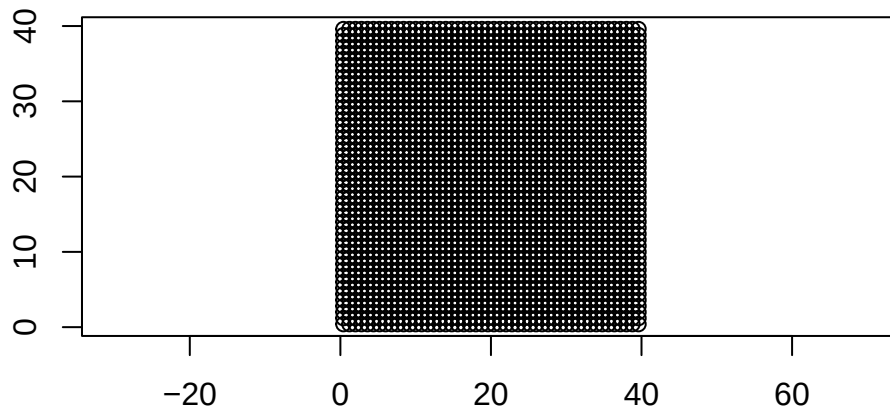
```
# Grid dimension
dim = 40

# Number of points to simulate
n = 2500
nrw = 50
ncl = 50

# Defining the domain size
sfc = st_sfc(
  st_polygon(list(rbind(c(0,0), c(dim,0), c(dim,dim), c(0,dim), c(0,0))))
)
```

```
# Simulating the coordinates
coordinates = st_make_grid(sfc, n=c(nrw,ncl), what = "centers")

plot(coordinates, axes = TRUE)
```



Capturing different regions inside the polygon

```
# Defining the circle 1
center1 <- c(33,8)
radius1 <- 5
circle1 <- st_buffer(st_sfc(st_point(center1)), dist = radius1)

# Defining the circle 2
center2 <- c(8,33)
radius2 <- 5
circle2 <- st_buffer(st_sfc(st_point(center2)), dist = radius2)

# Defining the quadrangle
length_quad <- 9
corner_coordinates_1 <- list(
  rbind(
    c(3,3),
    c(3+length_quad,3),
    c(3+length_quad,3+length_quad),
    c(3,3+length_quad), c(3,3))
)

square <- st_polygon(corner_coordinates_1)
```

```

# Defining the triangle
corner_coordinates_2 <- list(rbind(c(28,31), c(38,31), c(33,38), c(28,31)))
triangle <- st_polygon(corner_coordinates_2)

# Defining center larger circle and smaller circle at the center
centers <- c(20,20)
radius_l <- 8
radius_s <- 5

large_circle <- st_buffer(st_sfc(st_point(centers)), dist = radius_l)
small_circle <- st_buffer(st_sfc(st_point(centers)), dist = radius_s)

# Create a ring by subtracting the smaller circle from the larger circle
ring <- st_difference(large_circle, small_circle)

```

Capturing the coordinates inside different regions

```

# Circle 1
circle_1_coords <- st_intersection(coordinates, circle1)
# Quadrangle
quad_coords <- st_intersection(coordinates, square)
# Circle 2
circle_2_coords <- st_intersection(coordinates, circle2)
# Triangle
tri_coords <- st_intersection(coordinates, triangle)
# Ring
ring_coords <- st_intersection(coordinates, ring)
# Circle inside the ring
circle_s_coords <- st_intersection(st_intersection(coordinates, small_circle))

```

Capturing the number of coordinates inside each region

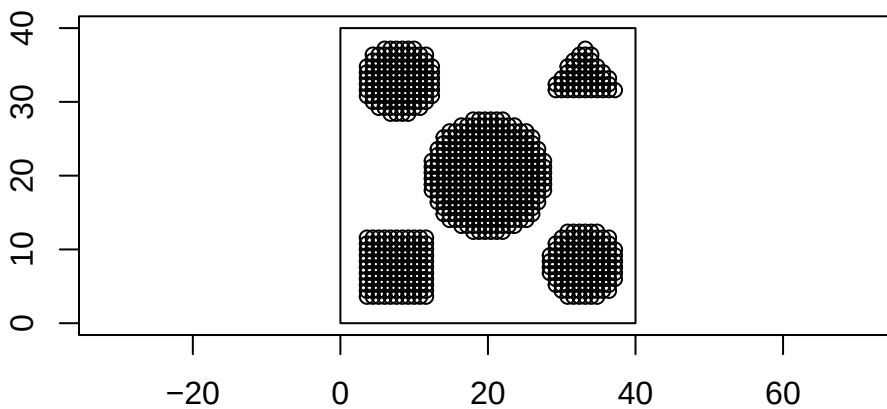
```

num_circle_1 <- length(circle_1_coords)
num_quad <- length(quad_coords)
num_circle_2 <- length(circle_2_coords)
num_tri <- length(tri_coords)
num_ring <- length(ring_coords)
num_circle_s <- length(circle_s_coords)

```

Plotting all the captured regions

```
cap_coords <- st_sf(  
  coords = c(  
    circle_1_coords,  
    circle_2_coords,  
    quad_coords,  
    tri_coords,  
    circle_s_coords,  
    ring_coords  
  )  
)  
plot(sfc, axes = TRUE)  
plot(cap_coords, add = TRUE)
```



Coordinates in the un-captured region (noise)

```
# Function to create an "sf object" of the union of the shapes  
union_sf <- function(sf_objects) {  
  result <- sf_objects[[1]]  
  for (i in 2:length(sf_objects)) {  
    result <- st_union(result, sf_objects[[i]])  
  }  
  return(result)  
}  
  
sf_list <- list(circle1, circle2, square, triangle, large_circle)
```

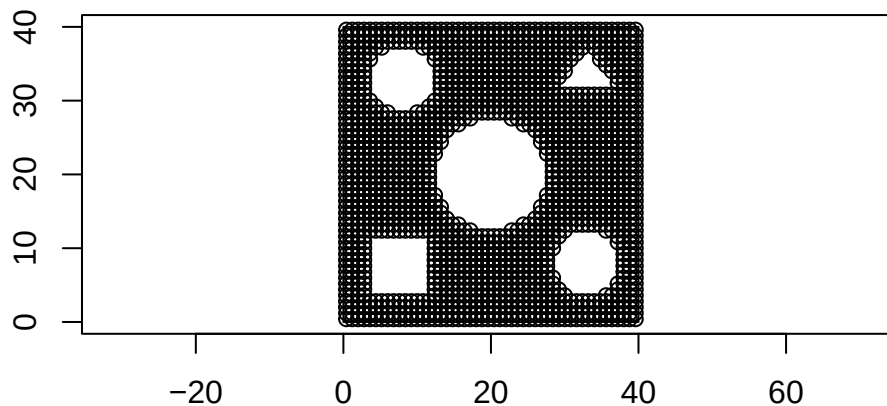
```

union_result <- union_sf(sf_list)

# Coordinates of the noise
non_coords <- st_sf(
  coords = st_difference(coordinates, union_result)
)

plot(sfc, axes = TRUE)
plot(non_coords, add = TRUE)

```



CAR

Function for simulations from spatial models

```

# Function to simulate for the selected regions
simulate_CAR <- function(n, coords, k, mu, tau_sq, eta){

  # Creating k-nearest neighbors
  nb_simulation <- knn2nb(knearneigh(coords, k))

  # Making the neighborhood structure un-directional
  nb_1_sym <- make.sym.nb(nb_simulation)

  # Weight matrix for simulation
  w_mat <- nb2mat(nb_1_sym, style = "W")
}

```

```

# Defining C matrix
c <- eta*w_mat

# Defining mean matrix
mu_mat <- rep(mu,n)

# Defining sigma matrix
# Defining sigma matrix
identity <- diag(seq(1,1,length=n))
sigma_mat <- tau_sq*solve(identity-c)

# Simulating from joint distribution of the variables from CAR model (MVN)
z_CAR <- mvrnorm(n=1, mu_mat, sigma_mat)
z <- as.vector(z_CAR)

return(z)
}

```

Function for simulating noise

```

simulate_noise <- function(n, coords, p, mu, sigma){

# Initiating mean and sigma
mu_mat <- rep(mu,p)

sigma_mat <- diag(rep(sigma, p))

# Simulating data from the multivariate normal distribution
noise <- mvrnorm(n=n,mu=mu_mat, Sigma = sigma_mat)
colnames(noise) <- paste0("Z",1:p)
data <- cbind(coords, noise)

return(data)
}

```

Simulate data

```

nperm <- 100
p <- 5      # Number of variables
k_sim <- 8   # Number of neighbors for simulation

# Model parameters for Square, Circle 1 and Ring
mu_1 <- 4
tau_sq_1 <- 1
eta_1 <- 0.8

# Model parameters for Square, Circle 1 and Ring
mu_2 <- 10
tau_sq_2 <- 1
eta_2 <- 0.8

# Model parameters for noise
mu_noise <- 0
sigma_noise <- 1

# List of different regions in region set 1
regions_1 <- list(
  st_sf(coords = circle_1_coords),
  st_sf(coords = quad_coords),
  st_sf(coords = ring_coords)
)

# Number of coordinates for each region in set 1
regions_1_num <- c(
  num_circle_1,
  num_quad,
  num_ring
)

# List of different regions in region set 2
regions_2 <- list(
  st_sf(coords = circle_2_coords),
  st_sf(coords = tri_coords),
  st_sf(coords = circle_s_coords)
)

# Number of coordinates for each shape in set 2
regions_2_num <- c(

```

```

    num_circle_2,
    num_tri,
    num_circle_s
  )

# Number of coordinates for noise
num_noise <- n - sum(regions_1_num, regions_2_num)

# Coordinates for simulating noise
noise_coords <- non_coords

results_list_1 <- list()
results_list_2 <- list()

for (i in 1:length(regions_1_num)) {

  # Simulating data for the regions in set 1
  sim_data_1 <- data.frame(
    lapply(
      1:p,
      function(x) simulate_CAR(
        regions_1_num[i],
        regions_1[[i]],
        k_sim, mu_1, tau_sq_1, eta_1
      )
    ),
    coords = regions_1[[i]]
  )

  names(sim_data_1)[1:p] <- paste0("Z", 1:p)
  results_list_1[[i]] <- sim_data_1

  # Simulating data for the regions in set 2

  sim_data_2 <- data.frame(
    lapply(
      1:p,
      function(x) simulate_CAR(
        regions_2_num[i],
        regions_2[[i]],
        k_sim, mu_2, tau_sq_2, eta_2
      )
    )
  )
}

```



```

    ),
    coords = regions_2[[i]]
  )

  names(sim_data_2)[1:p] <- paste0("Z", 1:p)
  results_list_2[[i]] <- sim_data_2
}

# Combining the two lists
results_list <- c(results_list_1, results_list_2)

# Name the result_list in the order of the simulated shapes
names(results_list) <- c(rep("clust1",3), rep("clust2",3))

# Convert the list to a data frame with a column for the name of the region
data_regions <- results_list |>
  bind_rows(.id = "sim_region")

# Simulate data for the noise coordinates and adding a region column
data_noise <- simulate_noise(num_noise, noise_coords, p, mu_noise, sigma_noise)

data_noise <- cbind(sim_region = "noise", data_noise)

# Convert data frame to an as sf object
data_regions <- st_as_sf(data_regions, sf_column_name = "coords")

# Merge noise and the clusters
data_regions <- rbind(data_regions, data_noise)

```

->

repSpat functions

Function for calculating MMD^2

```

compute_mmd_sq <- function(
  sample1_idx,
  sample2_idx,
  dist_matrix

```

```

    ) {

N <- length(sample1_idx)
M <- length(sample2_idx)

# First term: 1/N^2 sum K(xn, xn')
term1 <- sum(1/(sqrt(1+(dist_matrix[sample1_idx, sample1_idx])^2)))) / (N * N)

# Second term: -2/NM sum K(xn, xm')
term2 <- -2 * sum(1/(sqrt(1+(dist_matrix[sample1_idx, sample2_idx])^2)))) / (N * M)

# Third term: 1/M^2 sum K(x'm, x'm')
term3 <- sum(1/(sqrt(1+(dist_matrix[sample2_idx, sample2_idx])^2)))) / (M * M)

# MMD^2 value
mmd_sq <- term1 + term2 + term3
return(mmd_sq)
}

```

Function for creating blocks based on features inside each region (Using KMeans)

```

create_blocks <- function(data_regions, k_sim) {

# Initialize an empty list to blocks of each region
blk_data <- list()
regions <- unique(data_regions$region)

# For each region perform kmeans blocking
for (i in 1:length(regions)) {

  region_data <- data_regions[data_regions$region == regions[i],]
  num_blks <- floor(nrow(region_data)/k_sim)
  # If the cluster is smaller than the neighborhood size, use 1 bloc
  num_blks <- ifelse(num_blks == 0, 1, num_blks)

  df <- region_data[,2:(p+1)] |>
    st_drop_geometry()

  kmeans_result <- kmeans(df, num_blks)$cluster

```

```

    region_data <- cbind(region_data, polygon_id = kmeans_result)

    blk_data[[i]] <- region_data
  }

  combined_sf_df <- do.call(rbind, blk_data)

  return(combined_sf_df)
}

```

Function to perform block permutation

```

compute_perm_mmd2 <- function(cl1,
                              cl2,
                              dist_matrix,
                              df){

  # Saving the original order
  df$point <- 1:nrow(df)

  # Extracting rows such that observation of the smaller cluster is at top
  cluster_list <- list(cl1,cl2)
  sorted_list <- cluster_list[order(sapply(cluster_list, length))]

  df <- df[c(sorted_list[[1]], sorted_list[[2]]),]

  df <- df |>
    group_by(region, polygon_id) |>
    mutate(block_id = cur_group_id()) |>
    ungroup()

  # Saving the number of observations in of the smallest cluster
  n1 <- sum(df$region == unique(df$region)[1])

  blk_id1 <- c()
  n1_permuted <- 0

  while (n1_permuted < n1) {
    new_block <- sample(

```

```

    setdiff(
      unique(df$block_id),
      blk_id1),
    1
  )
  blk_id1 <- c(blk_id1, new_block)

  # The number of data points in the sampled blocks
  n1_permuted <- length(df$polygon_id[df$block_id %in% blk_id1])
}

df <- df %>%
  mutate(
    permuted_region = ifelse(
      block_id %in% blk_id1,
      unique(df$region)[1],
      unique(df$region)[2]
    )
  )

perm_mmd_sq <- compute_mmd_sq(
  df$point[df$permuted_region == unique(df$permuted_region)[1]],
  df$point[df$permuted_region == unique(df$permuted_region)[2]],
  dist_matrix
)
return(perm_mmd_sq)
}

```

Function to conduct pairwise MMD test

```

MMD_pairwise_test <- function(data_regions, dist_matrix, nperm) {

  # list to store pairwise results
  pairwise_results <- list()

  # Unique cluster pairs
  cluster_pairs <- combn(unique(data_regions$region), 2)

  for (i in 1:ncol(cluster_pairs)) {

```

```

cluster_1 <- which(data_regions$region == cluster_pairs[1, i])
cluster_2 <- which(data_regions$region == cluster_pairs[2, i])

obs_mmd_sq <- compute_mmd_sq(cluster_1, cluster_2, dist_matrix)

null_mmd_sq <- lapply(
  1:nperm,
  function(i){
    compute_perm_mmd2(cluster_1, cluster_2, dist_matrix, data_regions)
  }
) |> unlist()

p_value <- mean(null_mmd_sq >= obs_mmd_sq)
pairwise_results[[i]] <- c(cluster_pairs[1, i], cluster_pairs[2, i], p_value, obs_mmd_sq)
}

# combine list elements to a datarame
results_df <- as.data.frame(do.call(rbind, pairwise_results))
colnames(results_df) <- c("region_1", "region_2", "p_value", "obs_mmd_sq")

# Get the adjusted p-value
results_df$adj_p <- p.adjust(results_df$p_value, method = "BH")

return(results_df)
}

```

Apply repSpat

```

# Distance matrix on features
feature_distance <- data_regions[, 2:(p+1)] |>
  st_drop_geometry() |>
  dist() |>
  as.matrix()

locs <- st_coordinates(data_regions)

# Apply constrained clustering
nb <- knn2nb(knearneigh(
  as.data.frame(locs), k = k_sim)) %>%

```

```

nb2listw(style = "W")

neighbors <- listw2sn(nb)[,1:2]

constr.clust <- constr.hclust(
  d = as.dist(feature_distance), method = "ward.D2",
  links = neighbors, coords = locs
)

# Optimal number of clusters for constrained is 7
data_regions$region <- cutree(constr.clust, 7)

# Creating blocks using k-means
data_regions <- create_blocks(data_regions, k_sim)

test_results <- MMD_pairwise_test(data_regions, feature_distance, nperm)

# Identify cluster pairs which didn't reject null
# similar_pairs <- test_results[test_results$p_value >= 0.05, c("region_1", "region_2")]
similar_pairs <- test_results[test_results$p_value >= 0.05, c("region_1", "region_2", "obs_m

all_nodes <- unique(c(test_results$region_1, test_results$region_2))

# Convert the results to a graph
# g <- graph_from_data_frame(similar_pairs, directed = FALSE)

g <- graph_from_data_frame(similar_pairs, vertices = data.frame(name = all_nodes), directed =

# Obtain fully connected sub-graphs with a minimum of 3 nodes
cliques <- cliques(g, min = 3)
clique_nodes <- lapply(cliques, names)

# Re-partitioning the clusters
if (length(clique_nodes) != 0) {

  for (iter in 1:length(clique_nodes)) {
    data_regions$region <- replace(
      data_regions$region,
      data_regions$region %in% as.numeric(clique_nodes[[iter]]),
      letters[iter]
    )
  }
}

```

```
}
```

Print test results

```
head(test_results)
```

	region_1	region_2	p_value	obs_mmd_sq	adj_p
1	1	2	0.98	0.01927999	1
2	1	3	0.98	0.01814875	1
3	1	4	0.00	0.51377556	0
4	1	5	0.00	0.51516894	0
5	1	6	0.00	0.51891899	0
6	1	7	0.00	0.45718398	0

```
write.csv(test_results, "/Users/pratheepajeganathan/Library/CloudStorage/Dropbox/GitHub/HQP-")
```

Print the graph

```
png("graph_plot.png",
    width = 7 * 300,
    height = 5 * 300,
    res = 300)

# create layout first
lay <- layout_with_fr(g)

# spread nodes farther apart
lay <- lay * 1.8

plot(
  g,
  layout = lay,
  vertex.size = 40,
  vertex.color = "lightblue",
  vertex.label.cex = 1.2,
  edge.width = 2,
  edge.curved = 0.2,
```

```

    edge.label = round(E(g)$obs_mmd_sq, 2),
    edge.label.cex = 1,
    edge.label.dist = 0.5
  )
dev.off()

```

pdf
2

Plot repSpat results

```

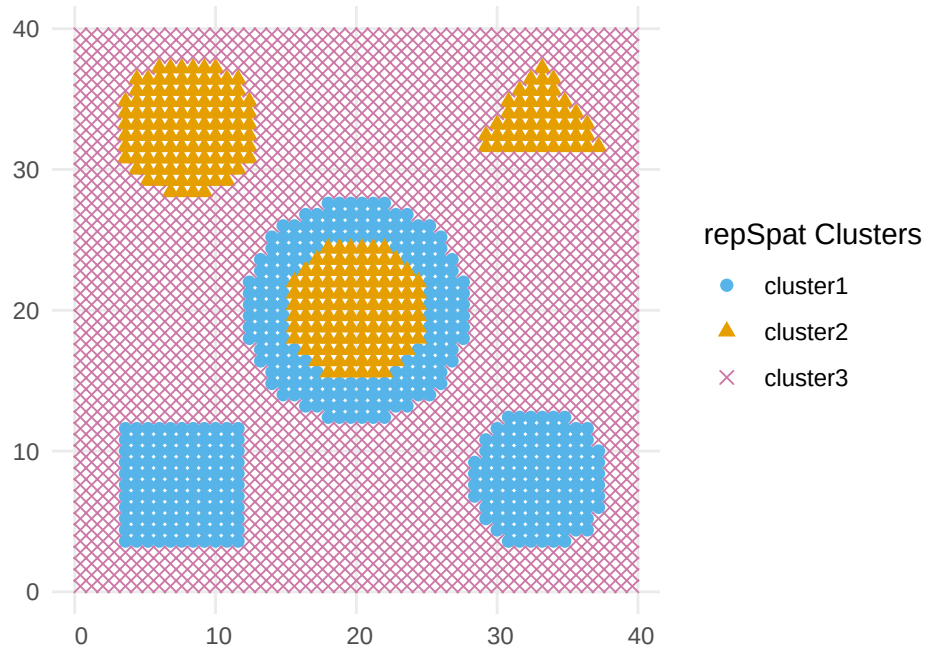
cb_palette <- c("#56B4E9", "#E69F00", "#CC79A7", "#009E73", "#0072B2", "#D55E00", "#F0E442")

data_regions$region <- recode(data_regions$region,
                              "a" = "cluster2",
                              "b" = "cluster1",
                              "7" = "cluster3")

p1 <- ggplot() +
  geom_sf(
    data = data_regions,
    aes(
      geometry = coords,
      color = region,
      shape = region
    ),
    size = 2
  ) +
  scale_color_manual(values = cb_palette) +
  scale_shape_manual(values = c(
    cluster1 = 16, # circle
    cluster2 = 17, # triangle
    cluster3 = 4   # x
  )) +
  theme_minimal() +
  labs(
    color = "repSpat Clusters",
    shape = "repSpat Clusters"
  )

```


p1



```
ggsave("repSpat_results.png", plot = p1, width = 7, height = 5, units = "in")
```

Apply BANSKY

```
# # Convert the list to a data frame with a column for the name of the region
data_regions <- results_list |>
  bind_rows(.id = "sim_region")
#
# # Simulate data for the noise coordinates and adding a region column
data_noise <- simulate_noise(num_noise, noise_coors, p, mu_noise, sigma_noise)
data_noise <- cbind(sim_region = "noise", data_noise)

# # Convert data frame to an as sf object
data_regions<- st_as_sf(data_regions, sf_column_name = "coords")

# # Merge noise and the clusters
data_regions <- rbind(data_regions,data_noise)
```

```
# Create a spatial experiment object to apply Banksy
assay <- data_regions[, -1] %>% st_drop_geometry() %>% t()

locs <- st_coordinates(data_regions)

se <- SpatialExperiment(assay = list(counts = assay), spatialCoords = locs, colData = locs)

# Compute mean gene expression values and AGFs
se <- computeBanksy(se, assay_name = 'counts', compute_agf = TRUE, k_geom = k_sim)
```

Computing neighbors...

Spatial mode is kNN_median

Parameters: k_geom=8

Done

Computing neighbors...

Spatial mode is kNN_median

Parameters: k_geom=8

Done

Done

Centering

Done

```
# Cluster using kmeans with 3 cluster centers
se <- clusterBanksy(
  se, use_agf = TRUE, use_pcs = FALSE, lambda = 0.8,
  algo = 'kmeans', kmeans.centers = 3
)

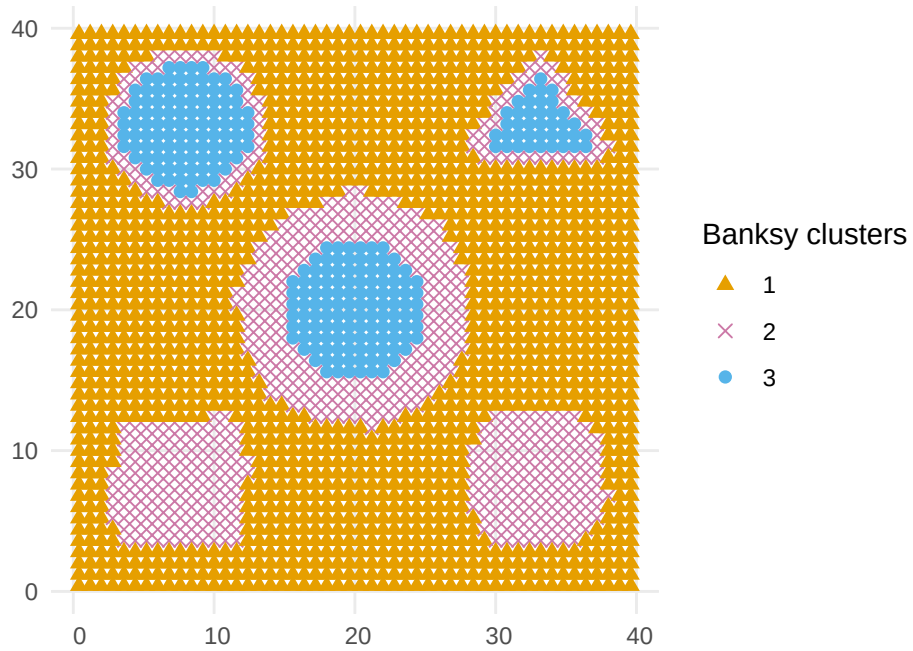
data_regions$`Banksy clusters` <- se$clust_M1_lam0.8_kmeans3
```

Plot Banksy results

```
cb_palette <- c("#E69F00", "#CC79A7", "#56B4E9", "#009E73", "#0072B2", "#D55E00", "#F0E442")

g_banksy <- ggplot() +
  geom_sf(
    data = data_regions,
    aes(
      geometry = coords,
      color = `Banksy clusters`,
      shape = `Banksy clusters`
    ),
    size = 2
  ) +
  theme_minimal() +
  scale_color_manual(values = cb_palette) +
  scale_shape_manual(values = c(
    "3" = 16, # circle
    "1" = 17, # triangle
    "2" = 4   # x
  )) +
  labs(
    color = "Banksy clusters",
    shape = "Banksy clusters"
  ) +
  theme(legend.position = "right")

g_banksy
```



```
ggsave("bansky_results.png", plot = g_bansky, width = 7, height = 5, units = "in")
```